

Senior AI Engineer / Decision Scientist Technical Assessment

48-hour take-home challenge

Challenge: Build an Intelligent Knowledge Assistant

Build a production-oriented AI application that can answer questions over a collection of unstructured documents. The solution should combine Retrieval-Augmented Generation (RAG) with an agentic workflow for questions that require multiple steps or capabilities.

Make your safe assumptions if any questions or doubts arise.

What You Need to Build

1. Document Ingestion & Knowledge Base

- Create a reusable pipeline to load documents, extract text, chunk content, generate embeddings, and store the resulting vectors.
- Preserve useful metadata such as document name, page number/source and chunk identifier.
- The ingestion process should be separate from the query-time application and should not re-process the entire corpus for every question.

2. RAG-Based Question Answering

- Retrieve relevant content and use an LLM to generate an answer grounded in the retrieved evidence.
- Return source/citation information with the answer (for example, document and page).
- Handle questions for which the knowledge base does not contain sufficient information. The system should avoid fabricating an answer.

3. Agentic Workflow

- Implement an agentic workflow for questions that require multiple steps, multiple retrievals, comparison, or tool use.
- The system should determine when a simple RAG flow is sufficient versus when an agentic flow is appropriate.
- Use any suitable framework or custom orchestration. Framework choice is yours; explain the reasoning behind it.

4. API & Application Interface

- Expose the solution through a REST API (FastAPI or equivalent).
- Provide a clear request/response contract for asking questions.

- The response should include the answer, sources, and an indication of the workflow used (where applicable).

Containerization

- Provide a `Dockerfile` and/or `docker-compose.yml` to containerize the solution.
- The application should be runnable with a single `docker-compose up` command (or equivalent).
- Include any necessary environment variable configuration via `.env.example`.

5. Evaluation & Testing

- Create a small evaluation set (approximately 10–15 questions) covering straightforward questions, multi-step questions, ambiguous questions, and questions with no answer in the corpus.
- Describe how you assess retrieval quality, answer quality/grounding, and agent/workflow behaviour.
- Include unit tests for key components and at least one end-to-end/API test.

Engineering Expectations

Treat this as a small production-oriented engineering exercise, not a notebook-only prototype. We are interested in your engineering judgement as much as the final output.

Area	What we expect to see
Python & Engineering	Clean, modular, maintainable Python; configuration management; error handling; logging; sensible abstractions.
RAG	Reasonable chunking/retrieval strategy, metadata, source attribution, grounded responses and failure handling.
Agentic AI	Purposeful tool/workflow design, appropriate routing, state/step handling and graceful failures.
System Design	Clear architecture and discussion of scalability, latency, reliability, security and cost trade-offs.
Production Readiness	API-based service, reproducible setup, tests and consideration of deployment/operational concerns.
Evaluation	A practical method to measure retrieval and answer quality and identify failure modes.

System Design

Include an architecture diagram showing the major components and data flow. Briefly explain your key design decisions, including:

- LLM and embedding model selection
 - Vector store / retrieval approach
 - Agent orchestration approach and when it is invoked
 - Scalability, latency and cost considerations
 - Security, secrets/configuration and failure handling
 - How you would monitor and evaluate the system in production
-

GitHub Submission

- Submit a link to a GitHub repository containing the complete solution.
 - Include a README with: problem understanding, architecture, setup/run instructions, API usage, design decisions, evaluation results, limitations and production improvements.
 - Include an `.env.example` or equivalent. Do not commit API keys, credentials or other secrets.
 - Recommended structure: `src/` (application code), `tests/`, `scripts/`, `configs/`, `README.md`, architecture diagram, and dependency/environment files. You may organise the repository differently if you have a good reason.
-

Demo Submission

- 1 Simple UI (Recommended): Build a lightweight interface using Streamlit, Gradio, or Chainlit that demonstrates your solution. The UI should allow a user to ask questions and see answers with sources. *This is not a frontend assessment — keep it simple.*
 - 2 Demo Recording (Required): Record a 5-minute max screen recording (Loom, OBS, or similar) walking through:
 - Your architecture and key design decisions (1 min)
 - Live demo of 3-4 questions covering: simple RAG, multi-step agentic, and an unanswerable question (3 min)
 - One thing you'd improve with more time (1 min)
-

Technology & AI-Assistance Policy

- You may use LLMs/AI coding assistants, documentation, open-source libraries and online resources.
 - You may choose your own LLM provider, vector database, embedding model and orchestration framework.
-

- The submitted solution must be your own work. During the interview, you should be able to explain the architecture, code, trade-offs and important implementation decisions.
 - We may ask you to modify or extend parts of the solution during the technical discussion.
-

Provided Document Corpus

To ensure consistency across all candidates, the following sample document corpus is provided. You must use this corpus as the primary knowledge base for your solution. You may add supplementary documents, but your evaluation set and demo must be based on these documents.

Note: These documents will be shared as a ZIP file alongside this assessment.

#	Document	Description
1	RAG_Architecture_Patterns.pdf	RAG design patterns, chunking strategies, retrieval methods, hybrid search approaches
2	Vector_Database_Comparison.pdf	Comparative analysis of FAISS, Pinecone, Weaviate, pgvector, and Chroma — performance, cost, and use cases
3	Agentic_AI_Frameworks.pdf	LangChain, LangGraph, CrewAI, AutoGen — comparing orchestration approaches, tool use, and multi-agent patterns

Why this corpus?

- Enables **cross-document retrieval** (e.g., “Compare vector databases for production RAG systems”)
 - Enables **multi-step reasoning** (e.g., “What security considerations should I address when deploying a RAG system at scale?”)
 - Enables **unanswerable question testing** (questions outside these 6 topics should be handled gracefully)
 - Is **domain-relevant** to the role you are applying for
-

Submission Checklist

- ☐ Working ingestion pipeline
- ☐ Working RAG workflow with source attribution
- ☐ Agentic workflow for multi-step questions
- ☐ REST API
- ☐ Evaluation set and evaluation approach
- ☐ Unit/end-to-end tests
- ☐ Architecture diagram

- ☐ README and reproducible setup instructions
- ☐ Dockerfile / docker-compose.yml for containerised deployment
- ☐ No secrets committed to the repository
- ☐ UI Demo recording

Evaluator Scoring Rubric

The following rubric will be used to evaluate submissions. Total: 100 points.

Note: This section is for evaluators. You may choose not to share this with candidates, or share it to help them understand weighting priorities.

Category	Points	What We're Evaluating
Functional Correctness	20	Ingestion pipeline works end-to-end; RAG returns relevant, grounded answers; agentic workflow handles multi-step questions; API endpoints function correctly with proper request/response contracts
ML / AI Quality	20	Chunking and embedding strategy; retrieval relevance and precision; answer grounding and hallucination avoidance; agent routing logic; prompt design and optimisation
Engineering Quality	20	Clean, modular, maintainable code; proper abstractions and separation of concerns; error handling and logging; configuration management; code readability and style
Architecture & System Design	15	Architecture diagram clarity; rationale for design decisions (LLM, vector store, orchestration); scalability, latency, and cost trade-offs; component separation and data flow
Evaluation Approach	10	Quality and diversity of evaluation question set; coverage of edge cases (ambiguous, unanswerable); retrieval and answer quality metrics; failure mode identification
Testing	5	Unit test coverage for key components; at least one end-to-end/API test; test quality, assertions, and edge case coverage
Security & Responsible AI	5	No secrets committed; input validation and sanitisation; error message safety; data privacy considerations; prompt injection awareness
Documentation	5	README completeness; setup reproducibility; API documentation; design decision explanations; limitations and future improvements acknowledged
TOTAL	100	

Good luck — focus on clarity, engineering judgement, and a solution you can confidently explain.